

Pizzeria-Umsatz 2015: Schichtplan, Speisekarte, Aktion

Analyse der Kassendaten für Laura Martinez – drei Entscheidungen für Januar, belegt mit Zahlen

817.860 \$ Umsatz 2015 · 21.350 Bestellungen · 49.574 Pizzen

Laura muss im Januar drei Entscheidungen treffen

Neue Inhaberin einer Pizzeria in New Jersey, Kassenexport für das ganze Jahr 2015 – „Am wichtigsten sind mir konkrete Empfehlungen mit Zahlen.“

1

Schichtplan

Wann ist Stoßzeit, wann ist es ruhig?

Fragen 1–3

2

Speisekarte

Welche Pizzen können gestrichen werden? Zutaten verderben.

Fragen 4–6

3

Aktion

Wann und womit schwache Stunden stärken?

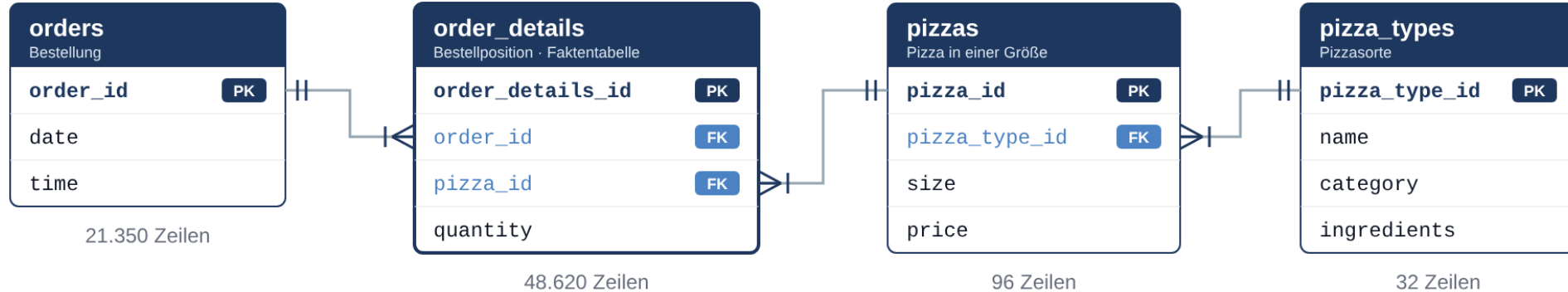
Fragen 5, 7, 8

8 Kernfragen: Kennzahlen · Monate · Wochentage & Uhrzeiten · Bestseller & Ladenhüter · Kategorien & Größen · Streichkandidaten · große Bestellungen · Aktion

4 Tabellen, 1 Jahr, 48.620 Bestellpositionen

Datenmodell: 4 Tabellen, 3 Beziehungen 1 : N

Pizza Place Sales 2015 (Kaggle) · Mitte: Faktentabelle order_details, 1 Zeile = 1 Bestellposition



Beziehungen (JOIN ... USING)

orders → order_details: 1 Bestellung → N Positionen

pizzas → order_details: 1 Pizza (Sorte + Größe) → N Positionen

pizza_types → pizzas: 1 Sorte → N Größen

Legende

PK Primärschlüssel **FK** Fremdschlüssel

||--}| links: genau 1 · rechts: 1 oder viele (N)

Geprüft: 0 fehlende Werte · 0 verwaiste Schlüssel · 0 doppelte IDs

Daten sind sauber – eine Tabelle `sales_lines` für das Dashboard

PRÜFUNG IN SQLITE ONLINE

- ✓ Zeilen gezählt: 21.350 · 48.620 · 96 · 32
- ✓ 0 fehlende Werte, 0 verwaiste Schlüssel, 0 doppelte IDs
- ✓ Zeitraum genau 01.01.–31.12.2015
- ✓ Typen mit `typeof()` : price = real, quantity = integer → `CAST` als Absicherung

KONTROLLE DER AUSWAHL

48.620

Zeilen = order_details
kein JOIN-Duplikat

817.860,05 \$

Umsatz = Frage 1

VIEW ÜBER ALLE 4 TABELLEN (★ BONUS) → EXPORT CSV

```
-- VIEW (★ Bonus)
CREATE VIEW sales_lines AS
SELECT
  o.order_id AS order_id,
  o.date AS order_date,
  o.time AS order_time,
  strftime('%m', o.date) AS month,
  CASE strftime('%w', o.date)
    WHEN '1' THEN 'Monday' WHEN '2' THEN 'Tuesday'
    WHEN '3' THEN 'Wednesday' WHEN '4' THEN 'Thursday'
    WHEN '5' THEN 'Friday' WHEN '6' THEN 'Saturday'
    WHEN '0' THEN 'Sunday'
  END AS weekday,
  strftime('%H', o.time) AS hour,
  pt.name AS pizza_name,
  pt.category AS category,
  p.size AS size,
  CAST(p.price AS REAL) AS unit_price,
  CAST(od.quantity AS INTEGER) AS quantity,
  CAST(p.price AS REAL) * CAST(od.quantity AS INTEGER) AS revenue
FROM orders o
JOIN order_details od USING (order_id)
JOIN pizzas p USING (pizza_id)
JOIN pizza_types pt USING (pizza_type_id);

-- Kontrolle
SELECT
  COUNT(*) AS row_count,
  ROUND(SUM(revenue), 2) AS total_revenue,
  SUM(quantity) AS pizzas_sold,
  COUNT(DISTINCT order_id) AS orders
FROM sales_lines;
```

1 Zeile = 1 Bestellposition, 12 Felder

818 Tsd. \$ Umsatz aus 21.350 Bestellungen

38,31 \$

Ø Bestellwert (817.860 / 21.350)

- **49.574** verkaufte Pizzen (Menge, nicht Zeilen)
- **Ø 2,32** Pizzen pro Bestellung (49.574 / 21.350)
- **Ø Bestellwert = Umsatz ÷ Bestellungen**, nicht AVG über Positionen

```
WITH base AS (  
  SELECT  
    SUM(CAST(p.price AS REAL) * CAST(od.quantity AS INTEGER)) AS total_revenue,  
    COUNT(DISTINCT od.order_id) AS total_orders,  
    SUM(CAST(od.quantity AS INTEGER)) AS pizzas_sold  
  FROM order_details od  
  JOIN pizzas p USING (pizza_id)  
)  
SELECT  
  ROUND(total_revenue, 2) AS total_revenue,  
  total_orders,  
  pizzas_sold,  
  ROUND(total_revenue / total_orders, 2) AS avg_order_value,  
  ROUND(CAST(pizzas_sold AS REAL) / total_orders, 2) AS avg_pizzas_per_order  
FROM base;
```

→ Für Laura: Ausgangsbasis – jede Maßnahme wird an diesen fünf Zahlen gemessen.

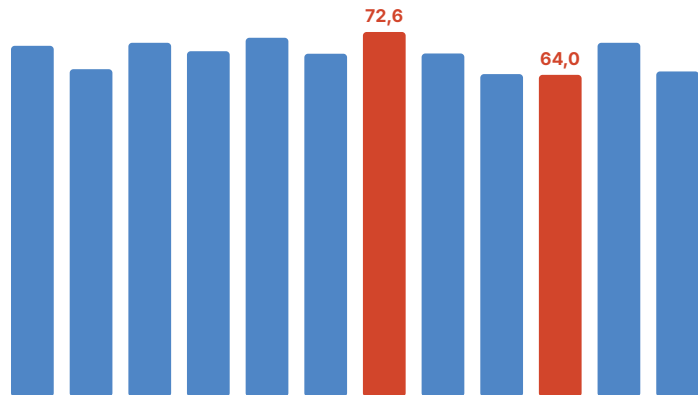
Kein Saisoneffekt: Monatsumsatz schwankt nur um 13 %

72.558 \$

stärkster Monat Juli · schwächster Oktober 64.028 \$

- Abstand 8.530 \$ = 13,3 % vom Oktober (8.530 / 64.028)
- ★ LAG: größter Anstieg November +9,9 %, Rückgang Dezember -8,1 %

Umsatz 2015 nach Monat, in Tsd. \$



```
WITH monthly AS (  
  SELECT  
    strftime('%m', o.date) AS month,  
    ROUND(SUM(CAST(p.price AS REAL) * CAST(od.quantity AS INTEGER)), 2) AS revenue,  
    COUNT(DISTINCT o.order_id) AS orders  
  FROM orders o  
  JOIN order_details od USING (order_id)  
  JOIN pizzas p USING (pizza_id)  
  GROUP BY month  
)  
SELECT  
  month,  
  revenue,  
  orders,  
  ROUND((revenue - LAG(revenue) OVER (ORDER BY month))  
    / LAG(revenue) OVER (ORDER BY month) * 100, 1) AS pct_vs_prev_month  
FROM monthly  
ORDER BY month;
```

→ Für Laura: Personal nach Wochentag und Uhrzeit planen, nicht nach Monat.

Freitag ist der stärkste Tag, 14–16 Uhr die Flaute

70,8

Bestellungen pro Freitag (3.538 / 50)

- Sonntag am ruhigsten: 50,5 pro Tag (2.624 / 52)
- Spitzen 12–14 Uhr (Stunden 12 und 13: 2.520 und 2.455 Bestellungen) und 17–19 Uhr (2.336 und 2.399)
- Vor 11 Uhr und ab 23 Uhr: nur 1, 8 und 28 Bestellungen im ganzen Jahr
- Falle: 48 Montage, 50 Freitage, sonst 52 → Durchschnitt pro Tag statt Summe

```
-- Q3a: Wochentage
SELECT
  CASE strftime('%w', date)
    WHEN '1' THEN 'Monday'
    WHEN '2' THEN 'Tuesday'
    WHEN '3' THEN 'Wednesday'
    WHEN '4' THEN 'Thursday'
    WHEN '5' THEN 'Friday'
    WHEN '6' THEN 'Saturday'
    WHEN '0' THEN 'Sunday'
  END AS weekday,
  COUNT(*) AS orders,
  COUNT(DISTINCT date) AS days,
  ROUND(CAST(COUNT(*) AS REAL) / COUNT(DISTINCT date), 1) AS avg_orders_per_day
FROM orders
GROUP BY weekday
ORDER BY avg_orders_per_day DESC;

-- Q3b: Stunden
SELECT
  strftime('%H', time) AS hour,
  COUNT(*) AS orders,
  ROUND(CAST(COUNT(*) AS REAL) / (SELECT COUNT(DISTINCT date) FROM orders), 1) AS avg_orders_per_day
FROM orders
GROUP BY hour
ORDER BY hour;
```

→ Für Laura: Schichten auf Mittag und Abend ausrichten, freitags verstärken; 14–16 Uhr ist das Fenster für die Aktion.

Thai Chicken bringt fast 4-mal so viel Umsatz wie Brie Carre

43.434 \$

Thai Chicken · Brie Carre 11.589 \$ ($43.434 / 11.589 = 3,7$)

- Umsatz- und Mengenlisten stimmen zu 3 von 5 überein; Menge-Spitze: Classic Deluxe 2.453 Stück
- Brie Carre ist in beiden Listen Letzter: 490 Stück

Umsatz 2015, in \$ · Top 5 blau, Flop 5 orange

Thai Chicken		43.434
Barbecue Chicken		42.768
California Chicken		41.410
Classic Deluxe		38.180
Spicy Italian		34.831
Spinach Pesto		15.596
Mediterranean		15.360
Spinach Supreme		15.278
Green Garden		13.956
Brie Carre		11.588

```
-- Top 5 Umsatz · Flop 5: ORDER BY revenue ASC · Menge: ORDER BY qty_sold
SELECT
  pt.name AS pizza_name,
  ROUND(SUM(CAST(p.price AS REAL) * CAST(od.quantity AS INTEGER)), 2) AS revenue,
  SUM(CAST(od.quantity AS INTEGER)) AS qty_sold
FROM order_details od
JOIN pizzas p USING (pizza_id)
JOIN pizza_types pt USING (pizza_type_id)
GROUP BY pt.name
ORDER BY revenue DESC
LIMIT 5;
```

→ Für Laura: Die Flop-Sorten sind Kandidaten für die Streichung (Frage 6).

Größe L bringt fast die Hälfte des Umsatzes: 45,9 %

45,9 %

Umsatzanteil Größe L (375.319 \$, 18.956 Stück)

- L + M = 76,4 %; XL + XXL zusammen nur 1,8 % (580 Stück)
- Kategorien fast gleich: Classic 26,9 · Supreme 25,5 · Chicken 24,0 · Veggie 23,7 %
- Abstand stärkste – schwächste Kategorie: 3,2 Prozentpunkte

L		45,9 %
M		30,5 %
S		21,8 %
XL		1,7 %
XXL		0,1 %

```
-- Q5a: Kategorien - Anteil per Unterabfrage
SELECT
  pt.category AS category,
  ROUND(SUM(CAST(p.price AS REAL) * CAST(od.quantity AS INTEGER)), 2) AS revenue,
  ROUND(
    SUM(CAST(p.price AS REAL) * CAST(od.quantity AS INTEGER)) * 100.0
    / (SELECT SUM(CAST(p2.price AS REAL) * CAST(od2.quantity AS INTEGER))
      FROM order_details od2
      JOIN pizzas p2 USING (pizza_id))
    , 1) AS revenue_share_pct
FROM order_details od
JOIN pizzas p USING (pizza_id)
JOIN pizza_types pt USING (pizza_type_id)
GROUP BY pt.category
ORDER BY revenue DESC;

-- Q5b *: Größen - Anteil mit SUM() OVER ()
WITH by_size AS (
  SELECT
    p.size AS size,
    SUM(CAST(p.price AS REAL) * CAST(od.quantity AS INTEGER)) AS revenue,
    SUM(CAST(od.quantity AS INTEGER)) AS qty_sold
  FROM order_details od
  JOIN pizzas p USING (pizza_id)
  GROUP BY p.size
)
SELECT
  size,
  ROUND(revenue, 2) AS revenue,
  qty_sold,
  ROUND(revenue * 100.0 / SUM(revenue) OVER (), 1) AS revenue_share_pct
FROM by_size
ORDER BY revenue DESC;
```

→ Für Laura: Über die Karte nach Sorte entscheiden, nicht nach Kategorie; die Aktion auf Größe L ausrichten.

4 von 32 Sorten: unter 1,9 % Umsatz und unter 1.000 Stück

4 Sorten

zum Streichen · 3 weitere beobachten

- Brie Carre 1,42 % (490 Stück) · Green Garden 1,71 %
• Spinach Supreme 1,87 % · Mediterranean 1,88 %
- HAVING: nur Sorten mit Anteil < 2 % (7 Sorten)
- CASE: < 1,9 % **und** < 1.000 Stück → 'remove', sonst 'watch'
- Beobachten: Spinach Pesto, Calabrese, Italian Vegetables (1,91–1,96 %)

```
-- Q6a: Kurzliste (HAVING) und Entscheidung (CASE)
WITH by_type AS (
  SELECT
    pt.name AS pizza_name,
    ROUND(SUM(CAST(p.price AS REAL) * CAST(od.quantity AS INTEGER)), 2) AS revenue,
    SUM(CAST(od.quantity AS INTEGER)) AS qty_sold,
    ROUND(SUM(CAST(p.price AS REAL) * CAST(od.quantity AS INTEGER)) * 100.0
      / (SELECT SUM(CAST(p2.price AS REAL) * CAST(od2.quantity AS INTEGER))
        FROM order_details od2
        JOIN pizzas p2 USING (pizza_id)), 2) AS revenue_share_pct
  FROM order_details od
  JOIN pizzas p USING (pizza_id)
  JOIN pizza_types pt USING (pizza_type_id)
  GROUP BY pt.name
  HAVING revenue_share_pct < 2.0
)
SELECT
  pizza_name, revenue, qty_sold, revenue_share_pct,
  CASE
    WHEN revenue_share_pct < 1.9 AND qty_sold < 1000 THEN 'remove'
    ELSE 'watch'
  END AS decision
FROM by_type
ORDER BY revenue;
```

→ Für Laura: Diese 4 Sorten sind die Streichkandidaten – Grenzen bewusst gewählt für „3–5 Positionen“.

Streichen kostet höchstens 6,87 % Umsatz und spart 6 Zutaten

6,87 %

Umsatz der 4 Sorten: 56.183 \$ (★ cum_share_pct)

- ★ Kumuliert: Brie Carre 1,42 → + Green Garden 3,12
→ + Spinach Supreme 4,99 → + Mediterranean 6,87 %
- Zutaten: alle 6 Spalten = 1 → jede Zutat nur in einer Sorte (5 aus Brie Carre, 1 aus Mediterranean)
- Falle: ohne Kommas findet 'Prosciutto' auch „Prosciutto di San Daniele“

```
-- Q6b *: kumulierter Umsatzanteil
WITH by_type AS (
  SELECT
    pt.name AS pizza_name,
    SUM(CAST(p.price AS REAL) * CAST(od.quantity AS INTEGER)) AS revenue,
    SUM(CAST(od.quantity AS INTEGER)) AS qty_sold
  FROM order_details od
  JOIN pizzas p USING (pizza_id)
  JOIN pizza_types pt USING (pizza_type_id)
  GROUP BY pt.name
)
SELECT
  pizza_name,
  ROUND(revenue, 2) AS revenue,
  qty_sold,
  ROUND(revenue * 100.0 / SUM(revenue) OVER (), 2) AS revenue_share_pct,
  ROUND(SUM(revenue) OVER (ORDER BY revenue), 2) AS cum_revenue,
  ROUND(SUM(revenue) OVER (ORDER BY revenue) * 100.0 / SUM(revenue) OVER (), 2) AS cum_share_pct
FROM by_type
ORDER BY revenue
LIMIT 7;

-- Q6c: Zutaten, die nur in einer Sorte vorkommen
SELECT
  SUM(' ' || ingredients || ', ' LIKE '%, Brie Carre Cheese,%') AS brie_cheese,
  SUM(' ' || ingredients || ', ' LIKE '%, Prosciutto,%') AS prosciutto,
  SUM(' ' || ingredients || ', ' LIKE '%, Caramelized Onions,%') AS caramelized_onions,
  SUM(' ' || ingredients || ', ' LIKE '%, Pears,%') AS pears,
  SUM(' ' || ingredients || ', ' LIKE '%, Thyme,%') AS thyme,
  SUM(' ' || ingredients || ', ' LIKE '%, Plum Tomatoes,%') AS plum_tomatoes
FROM pizza_types;
```

→ Für Laura: 4 Sorten streichen: weniger Zutaten verderben, höchstens 6,87 % Umsatz betroffen.

4+ Pizzen: 18,2 % der Bestellungen, 39,4 % des Umsatzes

83,14 \$

Ø Bestellwert groß · normal 28,35 \$ (2,9-mal so hoch)

- 3.880 große Bestellungen bringen 322.571 \$
- Ebene gewechselt: erst pro Bestellung summieren (CTE), dann einteilen (CASE)
- **Ja, ein Firmenangebot lohnt sich zu prüfen** – wer bestellt, zeigen die Daten nicht

```
WITH order_level AS (  
  SELECT  
    od.order_id,  
    SUM(CAST(od.quantity AS INTEGER)) AS pizzas_in_order,  
    SUM(CAST(p.price AS REAL) * CAST(od.quantity AS INTEGER)) AS order_revenue  
  FROM order_details od  
  JOIN pizzas p USING (pizza_id)  
  GROUP BY od.order_id  
)  
SELECT  
  CASE  
    WHEN pizzas_in_order ≥ 4 THEN 'large (4+)'  
    ELSE 'regular (1-3)'  
  END AS order_size,  
  COUNT(*) AS orders,  
  ROUND(COUNT(*) * 100.0  
        / (SELECT COUNT(*) FROM order_level), 1) AS orders_share_pct,  
  ROUND(SUM(order_revenue), 2) AS revenue,  
  ROUND(SUM(order_revenue) * 100.0  
        / (SELECT SUM(order_revenue) FROM order_level), 1) AS revenue_share_pct,  
  ROUND(AVG(order_revenue), 2) AS avg_order_value  
FROM order_level  
GROUP BY order_size;
```

→ Für Laura: Ein Angebot für Gruppen und Firmen testen – große Bestellungen tragen fast 40 % des Umsatzes.

Nachmittags-Deal 14–16 Uhr: +11.219 \$ pro Jahr bei +10 %

+11.219 \$

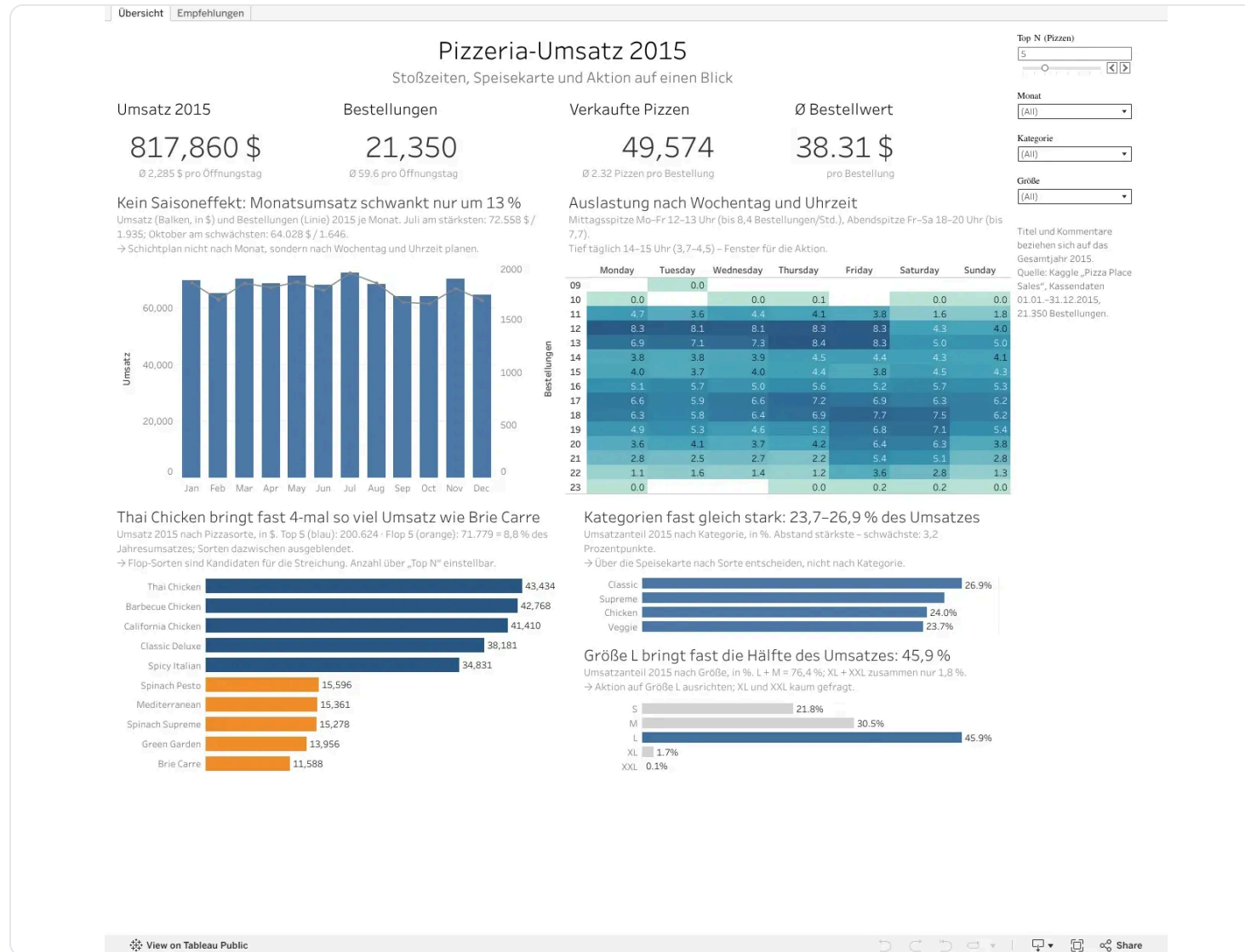
pro Jahr bei +10 % im Zeitfenster ($112.194 \times 0,10$)

- Fenster 14–16 Uhr heute: 112.194 \$ = 13,7 % des Umsatzes, 2.940 Bestellungen
- 313,39 \$ pro Tag ($112.194 / 358$) – Flaute an allen Wochentagen
- Vorschlag: täglich 14–16 Uhr Pizza L zum Preis von M (L = 45,9 % des Umsatzes)

```
-- Q8: Zeitfenster 14:00-15:59
WITH window_sales AS (
  SELECT
    SUM(CAST(p.price AS REAL) * CAST(od.quantity AS INTEGER)) AS revenue,
    COUNT(DISTINCT o.order_id) AS orders,
    COUNT(DISTINCT o.date) AS days
  FROM orders o
  JOIN order_details od USING (order_id)
  JOIN pizzas p USING (pizza_id)
  WHERE strftime('%H', o.time) IN ('14', '15')
)
SELECT
  ROUND(revenue, 2) AS window_revenue,
  orders,
  days,
  ROUND(revenue * 100.0
    / (SELECT SUM(CAST(p2.price AS REAL) * CAST(od2.quantity AS INTEGER))
      FROM order_details od2
      JOIN pizzas p2 USING (pizza_id)), 1) AS revenue_share_pct,
  ROUND(revenue / days, 2) AS revenue_per_day,
  ROUND(revenue * 0.10, 2) AS plus_10pct
FROM window_sales;
```

→ Für Laura: Aktion 4–6 Wochen testen – der Rabatt kostet Marge, der Nettoeffekt ist offen.

Laura kann selbst filtern: Monat, Kategorie, Größe



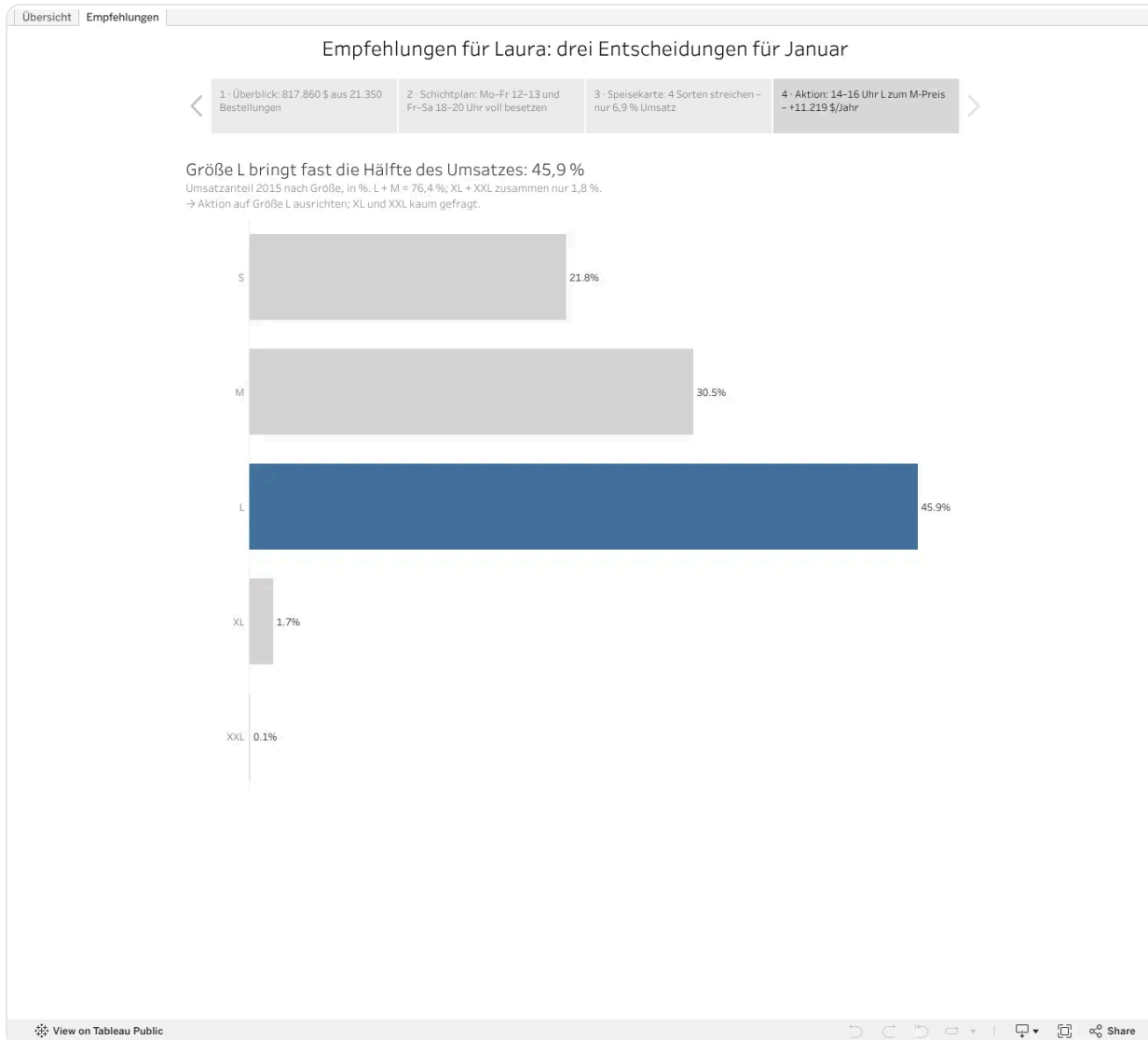
LIVE ZEIGEN

1. Filter **Monat** → KPI und Heatmap ändern sich
2. Klick auf **Größe L** → alle Grafiken gefiltert (★ Aktion)
3. Parameter **Top N 5** → 3 (★)
4. Story „**Empfehlungen**“: 4 Schritte (★)

Berechnete Felder: Ø pro Tag, Umsatzanteil (TOTAL), Bestellungen pro Wochentag (FIXED), Rang (RANK)

➔ Vollbild in Tableau Public

Vier Schritte vom Überblick zu Lauras Entscheidungen



SCHRITTE DER STORY

1. Überblick – KPI, Trend, Heatmap, Ranking
2. Schichtplan – Heatmap Wochentag × Uhrzeit
3. Speisekarte – Top/Flop mit Parameter Top N
4. Aktion – Umsatzanteil Größe L

➤ Vollbild in Tableau Public

Drei Dinge, die die Daten klar zeigen

70,8 / 50,5

Auslastung ist planbar

Bestellungen pro Freitag vs. Sonntag; Spitzen täglich mittags und abends, Flaute 14–16 Uhr.

45,9 %

Umsatz ist konzentriert

Größe L trägt fast die Hälfte; die 4 schwächsten Sorten nur 6,87 %.

39,4 %

Große Bestellungen zählen

18,2 % der Bestellungen mit 4+ Pizzen bringen fast 40 % des Umsatzes.

Drei Entscheidungen für Januar – jede mit Zahl

1

Schichtplan

Personal auf **12–14 Uhr** und **17–19 Uhr** ausrichten, **freitags** verstärken (70,8 Bestellungen/Tag), Fr–Sa abends bis 20 Uhr. 14–16 Uhr und sonntags (50,5/Tag) kleinere Besetzung.

Vor 11 Uhr und ab 23 Uhr: 1, 8 und 28 Bestellungen im Jahr → Öffnungszeiten prüfen.

2

Speisekarte

Brie Carre, Green Garden, Spinach Supreme, Mediterranean streichen: höchstens **6,87 %** des Umsatzes (56.183 \$), **6 Zutaten** weniger im Einkauf.

Spinach Pesto, Calabrese, Italian Vegetables (1,91–1,96 %) beobachten.

3

Aktion

Nachmittags-Deal täglich 14–16 Uhr: Pizza L zum Preis von M. Bei +10 % im Fenster **+11.219 \$** pro Jahr.

4–6 Wochen testen; zusätzlich ein Firmenangebot prüfen (große Bestellungen = 39,4 % des Umsatzes).

Was die Daten nicht beantworten

GRENZEN

- Nur Umsatz, keine Kosten und Marge → Nettoeffekt der Aktion offen
- Nur ein Jahr → Saison und Trend nicht trennbar
- Keine Kunden- und Personaldaten → wer bestellt groß, wie viele Mitarbeiter je Stunde
- 6,87 % ist eine Obergrenze: ein Teil der Gäste wählt eine andere Sorte

NÄCHSTE SCHRITTE

- Aktion als Vorher/Nachher- oder A/B-Test messen (4–6 Wochen)
- Uhrzeit und Wochentag großer Bestellungen auswerten → Firmenangebot
- Kosten und Verderb der Zutaten erfassen
- Personalstunden je Stunde mit Bestellungen abgleichen

VIELEN DANK!

Fragen?

GITHUB

github.com/SergiiVdovenkov/pizza-sales-analysis

DASHBOARD

public.tableau.com/views/pizza_dashboard_17908653158890

DATEN

kaggle.com/datasets/mysarahmadbhat/pizza-place-sales

Sergii Vdovenkov · Data Analytics & Engineering · Deutsche Tech Akademie